WEEK 1 · LESSON 4 OF 24 · HANDOUT

Build a Tiny GPT

Study time: **about 60 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=eMlx5fFNoYc — open the lesson online to watch inside the course.

🎓 **Go deeper:** Andrej Karpathy: Let's build GPT from scratch, in code — https://www.youtube.com/watch?v=kCc8FmEb1nY

🎓 **Go deeper:** 3Blue1Brown: Attention in transformers, step by step — https://www.youtube.com/watch?v=eMlx5fFNoYc

Your goal in this lesson: Build and train a small GPT-style transformer that writes text one character at a time, and connect every part of it to how real LLMs work.

The plan

GPT stands for **Generative Pre-trained Transformer**. We build the same architecture as the big models — just thousands of times smaller — and train it on our market sentences:

- **Tokens:** single characters (simple; real models use BPE).
- **Embeddings:** each token and each position gets a learned vector.
- **Transformer blocks:** masked self-attention + feed-forward, with residual connections and layer normalisation (you built these in Deep Learning & Neural Networks).
- **Output:** a score for every possible next character.
- **Training:** predict the next character at every position; loss = cross-entropy.

```
import random

random.seed(0)
names = ["Aline", "Eric", "Grace", "Jean", "Diane", "Patrick", "Claudine", "Samuel", "Ange",
"Didier"]
places = ["Kigali", "Huye", "Musanze", "Rubavu", "Nyagatare", "Rusizi", "Muhanga"]
goods = ["tomatoes", "beans", "bananas", "avocados", "maize", "potatoes", "milk", "coffee",
"rice", "cassava"]
verbs = ["sells", "buys", "carries", "cooks", "packs", "counts"]
times = ["in the morning", "at noon", "in the evening", "on Saturday", "after the rain",
"before school"]
moods = ["happy", "tired", "busy", "proud", "calm", "hungry"]
templates = [
    "{n} {v} {g} in {p} {t}.",
    "In {p}, {n} {v} {g} {t}.",
    "{n} is {f} because the {g} are fresh.",
    "{t}, {n} goes to the market in {p}.",
    "{n} pays for the {g} with mobile money.",
    "The price of {g} in {p} is low {t}.",
    "{n} tells {m} that the {g} from {p} are the best.",
    "{n} and {m} share {g} {t}.",
]

def sentence():
    n, m = random.sample(names, 2)
    s = random.choice(templates).format(n=n, m=m, v=random.choice(verbs),
g=random.choice(goods),
                                        p=random.choice(places), t=random.choice(times),
f=random.choice(moods))
    return s[0].upper() + s[1:]

text = "\n".join(sentence() for _ in range(4000))
print(len(text), "characters")
print(text[:250])
```

The model

```
import torch
import torch.nn as nn
import torch.nn.functional as F

device = "cuda" if torch.cuda.is_available() else "cpu"
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for c, i in stoi.items()}
encode = lambda s: [stoi[c] for c in s]
decode = lambda ids: "".join(itos[i] for i in ids)

data = torch.tensor(encode(text), dtype=torch.long)
split = int(0.9 * len(data))
train_data, val_data = data[:split], data[split:]

BLOCK = 64          # how many characters the model can look back at (its context window)

def get_batch(source, batch_size=32):
    starts = torch.randint(len(source) - BLOCK - 1, (batch_size,))
    x = torch.stack([source[s:s + BLOCK] for s in starts])
    y = torch.stack([source[s + 1:s + BLOCK + 1] for s in starts])   # the next character at
every position
    return x.to(device), y.to(device)

class Block(nn.Module):
    # One transformer block: masked self-attention + feed-forward, with residuals.
    def __init__(self, d, heads):
        super().__init__()
        self.ln1, self.ln2 = nn.LayerNorm(d), nn.LayerNorm(d)
        self.attn = nn.MultiheadAttention(d, heads, batch_first=True)
        self.ff = nn.Sequential(nn.Linear(d, 4 * d), nn.GELU(), nn.Linear(4 * d, d))
        mask = torch.triu(torch.ones(BLOCK, BLOCK, dtype=torch.bool), diagonal=1)
        self.register_buffer("mask", mask)              # True = may NOT look (the future)

    def forward(self, x):
        T = x.shape[1]
        h = self.ln1(x)
        a, _ = self.attn(h, h, h, attn_mask=self.mask[:T, :T], need_weights=False)
        x = x + a
        return x + self.ff(self.ln2(x))

class TinyGPT(nn.Module):
    def __init__(self, vocab, d=96, heads=4, layers=3):
        super().__init__()
        self.tok = nn.Embedding(vocab, d)
        self.pos = nn.Embedding(BLOCK, d)
        self.blocks = nn.Sequential(*[Block(d, heads) for _ in range(layers)])
        self.ln = nn.LayerNorm(d)
        self.head = nn.Linear(d, vocab)

    def forward(self, idx):
        T = idx.shape[1]
        x = self.tok(idx) + self.pos(torch.arange(T, device=idx.device))
        return self.head(self.ln(self.blocks(x)))       # logits: a score for every next
character

    @torch.no_grad()
    def generate(self, idx, n_new, temperature=1.0, top_k=None):
        for _ in range(n_new):
            logits = self(idx[:, -BLOCK:])[:, -1, :] / temperature
```

```
            if top_k is not None:
                cutoff = torch.topk(logits, top_k).values[:, [-1]]
                logits[logits < cutoff] = -float("inf")
            next_id = torch.multinomial(F.softmax(logits, dim=-1), num_samples=1)
            idx = torch.cat([idx, next_id], dim=1)
        return idx

@torch.no_grad()
def estimate_loss(model, steps=20):
    model.eval()
    out = {}
    for name, source in [("train", train_data), ("val", val_data)]:
        losses = []
        for _ in range(steps):
            x, y = get_batch(source)
            logits = model(x)
            losses.append(F.cross_entropy(logits.reshape(-1, logits.shape[-1]),
y.reshape(-1)).item())
        out[name] = sum(losses) / len(losses)
    model.train()
    return out

def train_gpt(steps=1500, seed=0):
    torch.manual_seed(seed)
    model = TinyGPT(len(chars)).to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=3e-3)
    for step in range(steps + 1):
        if step % 500 == 0:
            l = estimate_loss(model)
            print(f"step {step:4d}: train loss {l['train']:.3f}, validation loss
{l['val']:.3f}")
        if step == steps:
            break
        x, y = get_batch(train_data)
        logits = model(x)
        loss = F.cross_entropy(logits.reshape(-1, logits.shape[-1]), y.reshape(-1))
        opt.zero_grad()
        loss.backward()
        opt.step()
    return model

print("vocabulary:", len(chars), "characters")
```

Three details matter:

- **The causal mask** (`torch.triu`): each position may only look at **earlier** characters. Without it, the model could cheat by reading the answer during training.
- **Training targets** are the input shifted by one: for "Grace", the model learns G→r, Gr→a, Gra→c… — every position is a training example.
- **`generate`** feeds the model's own output back in, one character at a time — exactly how chatbots write their answers.

Before training: nonsense

```
torch.manual_seed(0)
untrained = TinyGPT(len(chars)).to(device)
start = torch.tensor([encode("Grace")], device=device)
print(decode(untrained.generate(start, 80)[0].tolist()))
print("parameters:", sum(p.numel() for p in untrained.parameters()))
```

Random characters, from about **350,000** parameters. (GPT-3 had 175 **billion**.)

Train it

```
model = train_gpt(steps=1500)
```

The loss starts around **4** (close to random guessing among 44 characters: ln 44 ≈ 3.8) and falls to about **0.27**. On a Colab GPU this takes under a minute; on a CPU a few minutes. The loss gets so low because our text is very regular — real text is far less predictable. The training and validation losses stay close: the model is not memorising.

After training: it writes

```
torch.manual_seed(1)
start = torch.tensor([encode("Grace")], device=device)
print(decode(model.generate(start, 200, temperature=0.8)[0].tolist()))
```

You should see sentences such as "Grace sells beans in Huye in the morning." — real-looking sentences in the right patterns, learned purely by predicting the next character. Nobody wrote a rule about grammar.

Notice: fluent but invented

The model happily writes "Ange buys coffee in Rusizi on Saturday" — a sentence that **never appeared** in the training text. It learned the **pattern** of sentences, not a list of true facts. That is the root of **hallucination** in real LLMs: they generate what is **likely**, not what is **true**.

From tiny to large

	Our TinyGPT	A large modern LLM
Tokens	44 characters	~100,000+ sub-word tokens
Context	64 characters	100,000s of tokens
Layers × width	3 × 96	~100 × ~10,000
Parameters	~350 thousand	tens to hundreds of billions
Training text	~200 thousand characters	trillions of tokens
Training time	minutes on one GPU	months on thousands of GPUs

The **ideas** are the same; scale changes what the model can do.

Key terms

- **GPT** — a decoder-only transformer trained to predict the next token.
- **Causal mask** — stops each position from seeing future tokens.
- **Autoregressive generation** — feeding each generated token back as input.
- **Pretraining** — next-token training on large amounts of raw text.
- **Hallucination** — fluent output that is not true.

Remember this: A GPT is token and position embeddings, a stack of masked attention blocks and a next-token classifier — trained only to predict the next token, it learns the patterns of language.

Name: _____ Date: _____

Activity

Task 1

Generate with temperature 0.3, 0.8 and 1.5. Describe the differences.

Task 2

Train for 300 steps only. What does the text look like?

Task 3

Remove the causal mask (set `attn_mask=None`) and train again. Why does the training loss fall so fast, and why is the generated text bad?

Task 4

Change `layers=3` to `layers=1`. How do the loss and the text change?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does GPT stand for?

   A. General Purpose Tool
   B. Guided Prompt Template
   C. Generative Pre-trained Transformer
   D. Graph Processing Technique

2. What does the causal mask prevent?

   A. Overfitting
   B. Using a GPU
   C. Long sentences
   D. Each position seeing future tokens

3. What are the training targets for a GPT?

   A. The first token only
   B. Images
   C. The input shifted by one position: the next token everywhere
   D. Random labels

4. The untrained model's loss is close to 3.8 with 44 possible characters. Why?

   A. It is the learning rate
   B. It is the number of layers
   C. ln(44) ≈ 3.8: it is guessing among 44 options at random
   D. It is a bug

5. The model writes a sentence that never appeared in its training text. What does this show?

   A. It learned patterns, and can produce fluent but invented content
   B. The data was leaked
   C. It copied the data
   D. Training failed

6. What is autoregressive generation?

   A. Deleting tokens
   B. Feeding each generated token back in to predict the next
   C. Training on images
   D. Sorting outputs

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.